Epitope Prediction Using Graph Neural Networks

Sadnan Sanim
dept. of Computer Science
BRAC University
Dhaka, Bangladesh
sadnan.sanim@g.bracu.ac.bd

Nahid Shahriar Rafi
dept. of Computer Science
BRAC University
Dhaka, Bangladesh
nahid.shahriar.rafi@g.brac.ac.bd

Nawroz Haseen Tumul
dept. of Computer Science
BRAC University
Dhaka, Bangladesh
nawroz.haseen.tumul@g.bracu.ac.bd

Md. Mezbaul Habib Ullash
dept. of Computer Science
BRAC University
Dhaka, Bangladesh
mezbaul.habib.ullash@g.bracu.ac.bd

***Abstract*—Epitope prediction is currently one of the most studied subjects. It is a critical task as it enables the identification of antigenic regions that trigger immune responses. This is essential for vaccine design and therapeutic development. Traditional methods are costly and extremely time-consuming. So they severely limit the real-world applications. Machine learning methods using protein sequences are an improvement over traditional methods, but they lose out on not having structural information. In this project, we used and compared two graph neural network models (GraphSAGE and GraphEPN) for predicting epitopes. We have utilized node features from ESM embeddings, physicochemical properties, and structural information from protein graphs. We trained the models with Focal Loss, class weighting, and included validation splits to check for convergence. Within the scope of this project, we got satisfactory results. With GraphEPN, we achieved an F1 of 0.2209 and an AUROC of 0.5870, with 0.6677 accuracy. With GraphSage, we achieved similar results. We got 0.2128 F1, 0.6025 AUROC, and 0.6685 accuracy.**

***Index Terms*—Epitope prediction, graph neural networks, GraphSAGE, GraphEPN, B-cell epitopes, protein graphs, ESM embeddings, Focal Loss, class imbalance, Epitope3D dataset, vaccine design.**

I. Introduction

Epitope prediction is the identification of specific areas on antigens that are recognized by the immune system, for example, by B-cell or T-cell epitopes. This is crucial for better diagnostics, vaccines, and immunotherapy. This is important for the following reasons. Predicting epitopes experimentally is very costly and time-consuming. It often involves high-throughput assays like mass spectrometry, which would be ineffective for new diseases or for those involved in personalized medicine. The appropriate computer predictions could speed up these processes, possibly saving time and money that would be needed for the development of effective vaccines against disease entities like COVID-19 or Nipah virus. This could greatly improve global health by allowing for rapid reactions to disease emergence and more tailored treatments. []

II. Past Works

Recent epitope prediction research has promoted the field with the use of deep learning, structural modeling, and comprehensive tools, dealing with data paucity, conformational intricateness, and model generalizability. The emphasis is on epitopes for both B-cell and T-cell and their performance in practical scenarios namely, dealing with unseen epitopes, class imbalance, and multi-allelic data. Choi and Kim (2024) suggest EpiGraph utilizes ESM-IF1 and ESM-2 embeddings to predict conformational B-cell epitopes [1]. Spatial clustering and structural/evolutionary information are captured with an AUROC value of 0.73 on the epitope3D dataset. It outperforms DiscoTope-3.0 and BepiPred-3.0. The advantages are that it handles graph homophily for epitope near-proximity and applies residual connections to minimize over-smoothing, making it robust for use on imbalanced datasets. Disadvantages are that it is antibody-agnostic and unable to predict antibody-specific epitopes. Also, performance drops with ESMFold-generated models due to structural errors. As a whole, EpiGraph is a powerful tool for general conformational prediction but is less favorable for personalized use. Besides, Jiang et al. (2024) say their AlphaEpi combines AlphaFold 3 structures and graph neural networks (GNNs) with a dynamic selector to aggregate sequence and structure information [2]. The model achieves an AUROC value of 0.766 on the BP3C50ID set and outperforms both CALIBER and GraphBepi. AlphaEpi is strong in noise resistance with dynamic selection and graph integration. It achieves at most 10 times better performance than baselines. AlphaEpi is reliant on AlphaFold 3's request limitation, which impacts scalability. It has a penchant for overfitting some datasets. It does poorly on heterogeneous epitopes. AlphaEpi excels best on conformational epitopes and should handle large-scale data better. Liu et al. (2024) present AsEP, a 1,723 antibody-antigen complex dataset, and WALLE, an ESM-2/GNNs hybrid for antibody-specific epitope prediction. WALLE has MCC 0.305 on epitope-to-antigen splits, outperforming MaSIF-site and

EpiPred [3]. Its performance is in the prediction of bipartite links. It improves interpretability and generalizability across epitope groupings. AsEP/WALLE tests AsEP but highlights generalizability shortcomings in deep learning for diverse epitopes. Xu et al. (2024) present ImmuneApp, a CNN-LSTM epitope prediction and immunopeptidome analysis model for HLA-I that was trained on 349,650 ligands [6]. The AUROC is 0.9576, better than NetMHCpan-4.1, and almost perfect on multitasking (e.g., neoantigen prioritization) and deconvolution of multi-allelic data, handling class imbalance nicely with attention. However, it is sensitive to data quality. Performance drops with allele coverage. It is also computationally expensive for large datasets. ImmuneApp is a strong performer on HLA-I tasks and fails on rare alleles or B-cell contexts. Nielsen et al. (2024) dissect the IMMREP23 competition, comparing TCR-epitope prediction methods [4]. Top-ranking methods such as IMW DETECT obtain AUC0.1 above 0.7 on observed pMHCs yet fall to 0.5 on unseen ones, exhibiting data leakage shortcomings. They point out negative sampling biases and the necessity for strong benchmarks while characterizing generalization gaps. The shortcomings are a focus on seen data, which limits understanding of new epitopes, and the failure to integrate structure. The study criticizes TCR prediction and stresses that current methods are ineffective for novel cases. Yan et al. (2024) introduce Next-Generation IEDB Tools that combine MHC binding and immunogenicity predictions [7]. It handles 1,000,000 amino acids efficiently. Moreover, it takes a leading position in unified pipelines and PEPMatch for proteome scanning and handling large analyses. Disadvantages are dependency on curated data. So it can be prone to produce biased predictions, and limited conformational epitope capability. IEDB is appropriate for high-throughput activity and less efficient for detailed structural predictions. Shabab et al. (2024) use immunoinformatics to design a multi-epitope vaccine against the Nipah virus [5]. They also make use of IEDB for epitope selection. They identify high immunogenicity candidates, which allows for feasible vaccine construction. The drawbacks are that they are dependent on linear epitopes and neglect conformational epitopes. It allows for fast vaccine prototyping. However, it is constrained by prediction error. These studies enhance epitope prediction using deep learning and structural information. Addresses data scarcity with large models and benchmarks. Advantages are higher precision (AlphaEpi's fusion) and interpretability (WALLE's associations). But there are still some issues. We have seen generalizing on unseen epitopes (IMMREP23), noise in predictions (EpiGraph), and scaling (ImmuneApp). Future research should focus on hybrid models for greater applicability.

Collatz et al. [9] introduce EpiDope, a deep neural network for linear B-cell epitope prediction that uses pretrained ELMo-style contextual embeddings with bidirectional LSTMs plus a task-specific embedding branch, trained on 24,610 redundancy-reduced IEDB epitope/non-epitope regions; it achieves ROC AUC approximately 0.67 and AUC10 percent 0.151 in 10-fold CV and remains best on an independent IEDB set (AUC 0.625, AUC10 percent 0.120), outperforming BepiPred-2.0, with strengths in contextual modeling, strict train–test separation, high top-end precision, and offline proteome-scale scanning, but limited by noisy short linear-epitope labels and exclusion of conformational/antibody-specific epitopes. Lu et al. [8] propose a structure-based conformational epitope predictor that fuses local GCNs on antigen spatial graphs (sequence, evolutionary, physicochemical, structural features) with attention BiLSTMs for global sequence context, reaching AUC ROC 0.804 and AUC PR 0.376, beating DiscoTope-2.0, EpiPred, and PECAN, and performing well on SARS-CoV-2 spike epitopes for multiple neutralizing antibody types, though it requires structures and is PSI-BLAST-intensive. Wang et al. [11] present GraphEPN, combining unsupervised VQ-VAE pretraining on SAbDab1323 with a graph transformer, achieving AUC 0.829/AUPRC 0.433 (SAbDab665 CV) and leading on Blind42 (AUC 0.765, AUPRC 0.196), but remaining structure-dependent and compute-heavy. Separately, Liu et al. [16] forecast urban traffic speeds from sparse taxi GPS by recovering missing link speeds via NMTF/SNMF-based imputation and then applying GraphSAGE on a road graph, reducing MAE/RMSE versus ARIMA, SVR, XGBoost, and DCRNN, while noting limits such as low-rank assumptions, taxi-biased coverage, and missing external factors.

Our research looks at problems found in existing studies, like poor generalization to unseen epitopes, data imbalance challenge, noise in prediction, etc. We tried to address those, but have not made any breakthrough. We have compared GraphSAGE and GraphEPN to evaluate which model works better for epitope prediction tasks. We used Graphein to make the graphs from the PDB files. We built graphs based on radius, implemented FocalLoss, and tracked validation to make our approach stronger and easier to understand. Ultimately, our work here is a stepping stone for us. We plan to use our experience from this project to make further improvements on this study in the future, potentially with an even broader scope.

III. Our Contribution

- We used GraphSAGE and GraphEPN models to predict epitopes.
- Combined ESM embeddings, physicochemical properties, and protein structure features.
- We built a training setup with FocalLoss, class weighting, early stopping, and validation splits.
- Handled data imbalance and tracked model performance during training.
- We used radius-based edge construction for protein graphs.
- Extracted node features with Graphein and DSSP.
- We compared GraphSAGE and GraphEPN on test datasets.
- Designed the system to be memory-efficient and scalable.
- Tuned thresholds to improve the F1-score.
- Created graphs showing loss (test/train/validation) and AUROC.

IV. Material and Methods

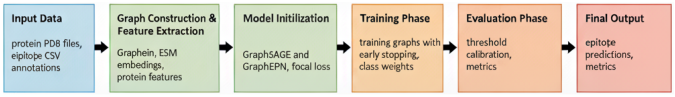

Fig. 1: General Pipeline of the project

The pipeline has five main parts: data collection, feature extraction, model design, training, and evaluation.

Data Collection and Preprocessing: Protein structures came from PDB files. Epitope labels came from CSV datasets for training and testing. Each protein was linked with its PDB ID. The data was cleaned. Proteins with very large or very small node counts were removed.

Feature Extraction: Protein graphs were built using Graphein. Amino acids were treated as nodes with 3D positions. Edges were added when residues were closer than 10 Å. Each node included:

- Normalized 3D coordinates
- Embeddings from the 33-layer ESM2 model
- Physicochemical values such as hydrophobicity, charge, and volume
- DSSP features like helix, sheet, loop, and relative surface area
- Flexibility from the B-factor

All features were joined into one node vector.

Graph Neural Network Model: The main model was GraphSAGE. It used several convolution layers. Batch normalization and dropout reduced overfitting. A classifier head predicted whether a residue was an epitope or a non-epitope. A learnable threshold improved the prediction.

Training Phase: Graphs were converted into PyTorch Geometric format. Training used FocalLoss with class weights to fix the imbalance. AdamW optimizer updated the weights. Early stopping was used to prevent overfitting.

Evaluation Phase: The model was tested on new protein graphs. Each residue got an epitope probability and a label. Threshold calibration improved the F1-score. Accuracy, precision, recall, F1-score, and AUROC were calculated. Results were stored for later use.

Output: The final output is epitope prediction for protein residues. These predictions can help in viral and immunology studies.

A. Dataset Description

For this project, the datasets used are sourced from the publicly available **Epitope3D** benchmark[1]. This resource provides curated mappings from protein 3D structures to annotated residue-level B-cell epitopes, widely used for evaluating computational immunology models. Both training and test data files are directly from Epitope3D and were not modified, ensuring benchmark reproducibility.

Dataset Properties

[1]https://biosig.lab.uq.edu.au/epitope3d/data

Dataset	Samples (Rows)	Features (Columns)	Columns
Training Set	180	2	PDB ID, Epitope List
Test Set	20	2	PDB ID, Epitope List

TABLE I: Properties of the Epitope3D datasets used in this study.

Feature Description:

- PDB ID: Unique identifier for the protein structure entry (e.g. "1CFI"), which is used to retrieve atomic coordinates from PDB files.
- Epitope List: Comma-separated annotations for residues predicted or labelled as epitopes. Each entry follows the format `residueid_residuename_chain`, like `10_VAL_A`.
  - *residueid:* Integer representing position in polypeptide chain
  - *residuename:* Three-letter amino acid code (e.g., LEU, VAL, GLN)
  - *chain:* Single character protein chain label (e.g., A, B)
- Sample from PDB file (used for graph construction):

```
ATOM 1 N LEU A 19 3.450 4.162 5.338
    1.00 41.69 N
ATOM 2 CA LEU A 19 2.632 2.965 5.521
    1.00 39.89 C
ATOM 3 C LEU A 19 1.959 2.934 6.883
    1.00 38.15 C
...
```

These atomic rows provide x, y, z coordinates and chemical information, forming the basis for residue-level graph representations.

Example from Test Dataset (CSV):

```
PDB ID  Epitope List
1CFI    "10_VAL_A, 5_LYS_A, 11_GLN_A, ......"
5UE3    "161_SER_A, 160_TYR_A, 167_VAL_A, .."
...
```

The Epitope3D dataset ensures broad coverage across protein families, contains expertly validated residue-level annotations, and is structured to facilitate direct integration into graph-based neural models. This makes it ideal for both method benchmarking and advancing model generalizability for unseen epitope surfaces.

B. Tools / Models / Algorithms Used

We utilized two advanced graph neural network models called GraphSAGE and GraphEPN. Furthermore, we chose graph neural networks due to the graph-like structure of proteins, where amino acid residues can be represented as nodes and their spatial properties as edges. We derived the protein structures from the PDB files. Furthermore, we implemented these models using the PyTorch Geometric library which provides efficient tools for our models. Graphs used for both of these models were created using radius-based

edges with a threshold value of 10 Å. Furthermore, the node features combine ESM-2 embeddings with physicochemical properties, coordinates, B-factors and DSSP-derived secondary structures. During training, we used FocalLoss to address the class imbalance between epitopes and non-epitopes present in our dataset along with AdamW optimizer and gradient clipping.

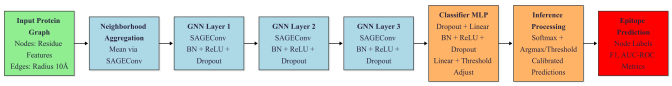

Fig. 2: GraphSAGE workflow diagram

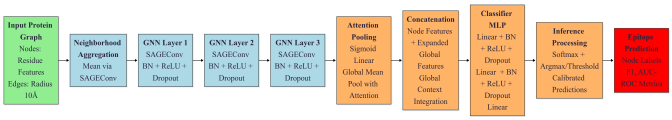

Fig. 3: GraphEPN workflow diagram

GraphSAGE or Graph Sample and Aggregate, is an inductive framework designed for generating node embeddings by sampling and aggregating features from a node's local neighbourhood. Hence, we selected this model due to its scalability for large graphs and ability to generalize to unseen nodes. Our `graphSAGE` model consists of 3 SAGEConv layers. The input dimension is 1291 which is derived from the node features with 3 normalized coordinates, 1280 ESM `embeddings` and 3 physicochemical properties. This architecture is `followed` by BatchNorm1d, ReLU activation. Finally the output dimension is set to 2 due to the task of binary classification. Our learnable epitope threshold further adjusts the epitope class logits for better precision recall balance. Our hyperparameters include a learning rate of 3e-4, weight decay of 3e-3, focal alpha of 0.6 and focal gamma of 1, epochs per graph of 40 with a patience of 20 for early stopping and a validation split of 0.1. These hyperparameters were selected to mitigate overfitting in deep layers, prevent parameter explosion and to increase focus on the relatively smaller number of epitopes while avoiding underfitting and promoting generalization on the training data.

GraphEPN or graph based epitope prediction network extends GraphSage with attention pooling, making it suitable for capturing long-range dependencies in protein structures. We also added a mechanism for weighted global pooling which may help the model work on sparse graphs. The model contains a similar architecture to our GraphSAGE model, with 3 SageConv layers along with ReLU and dropout of 0.2. Furthermore, our attention pooling consists of sigmoid activation on weights to get weighted global mean pooling. Finally our classifier consists of BatchNorm1d and dropouts of 0.4 and 0.2. Our hyperparameters include a lower learning rate of 3e-5 while maintaining parameters similar to GraphSAGE. Similar to GraphSAGE, we used FocalLoss as loss function and AdamW with gradient clipping.

C. Performance Evaluation

As we used FocalLoss as the primary loss function, which extends binary cross-entropy by allocating lower weights to common examples(non-epitopes) and higher weights to uncommon examples(epitopes). Furthermore, the loss function incorporates parameters alpha(0.25-0.6) to balance positive and negative classes and gamma (1.0) to increase focus on minority classes.

$$\mathrm{FL}(p) = \begin{cases} -\alpha\,(1-p)^{\gamma}\,\log(p), & \text{if } y = 1, \\ -(1-\alpha)\,p^{\gamma}\,\log\big(1-p\big), & \text{otherwise.} \end{cases}$$

Our dataset consists of the PDB files sourced from epitope3d, including our testing and training csv files. Furthermore, we implemented a 90/10 split with 10% of the testing data for validation split. We evaluate our results on the basis of whether our model was able to predict an epitope accurately.

V. Experimental Analysis

In this section, we will discuss the evaluation metrics obtained through our **Google Colab project.**[2]

Metric	GraphSAGE (Best)	GraphEPN (Best)
Accuracy	0.6685	0.6590
AUROC	0.6025	0.5869
Precision (Macro)	0.5314	0.5350
Recall (Macro)	0.5804	0.5917
F1-Score (Macro)	0.5014	0.5013
Precision (Epitope)	0.1374	0.1411
Recall (Epitope)	0.4716	0.5085
F1-Score (Epitope)	0.2128	0.2209
Best Threshold	0.50	0.50

TABLE II: Comparison of Best Results: GraphSAGE vs. GraphEPN

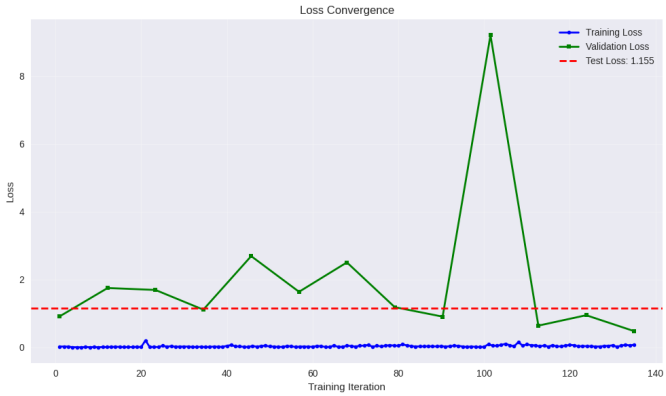

Fig. 4: GraphSAGE: Loss Convergence

[2]https://colab.research.google.com/drive/1rIG7HrbEujSi-EcLCbcPmBG1KeZtNjj4?usp=sharing

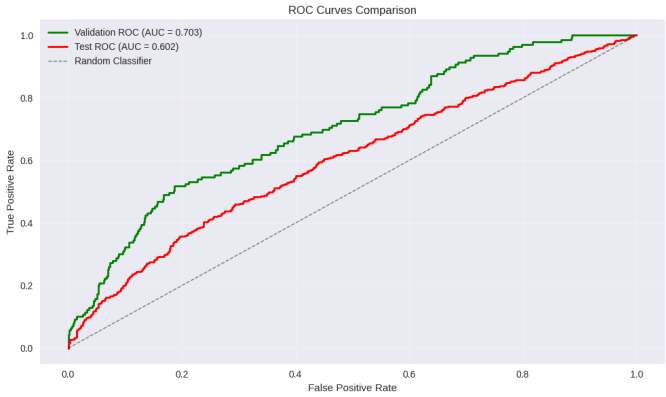

Fig. 5: GraphSAGE: ROC curves comparison

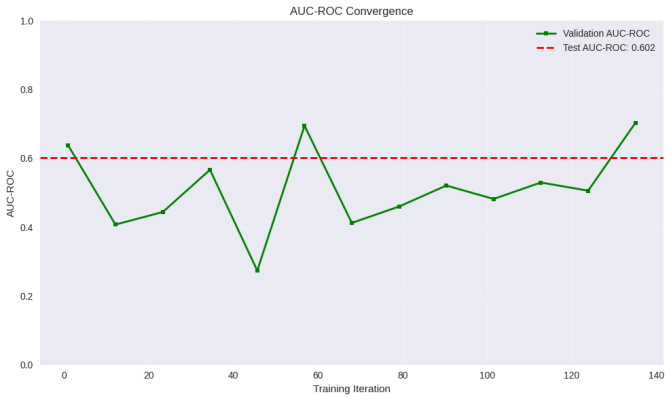

Fig. 6: GraphSAGE: AUROC convergence

These evaluation graphs were produced by graphSAGE using our best hyperparameters. The loss convergence graph displays the training, testing and validation loss. The training loss remains low and stable, while the validation loss varies periodically, indicating points of overfitting and recovery. The AUROC convergence graph tracks the score for validation set after every 10 intervals of training and is compared with the AUROC curve of test set. Validation AUC fluctuates between 0.3 to 0.7 across training iterations. This indicates variability in the model's ability to distinguish epitopes on unseen validation samples. Overall, the convergence behavior suggests that the model periodically overfits and recovers, however it was able to generalize on training data. Finally, the ROC Curves comparison graph presents the true positive rate against false positive rate for both validation and test sets. The validation ROC curve with AUC 0.703 outperforms the test ROC with AUC 0.602. These graphs provide further evidence that the model was able to generalize on the training data.

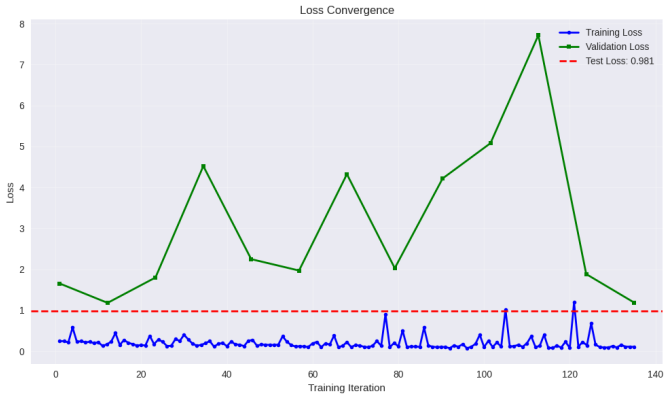

Fig. 7: GraphEPN: Loss Convergence

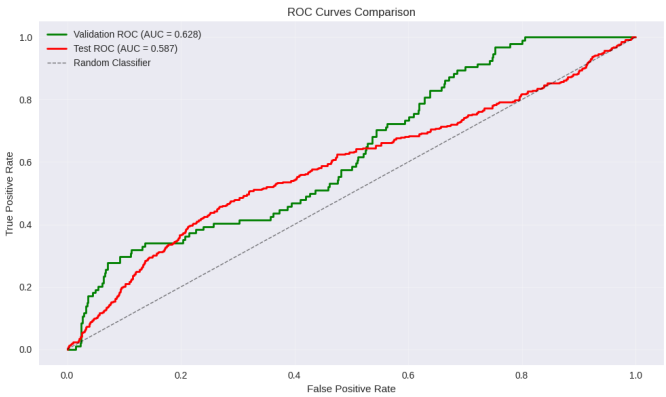

Fig. 8: GraphEPN: ROC Curves Comparison

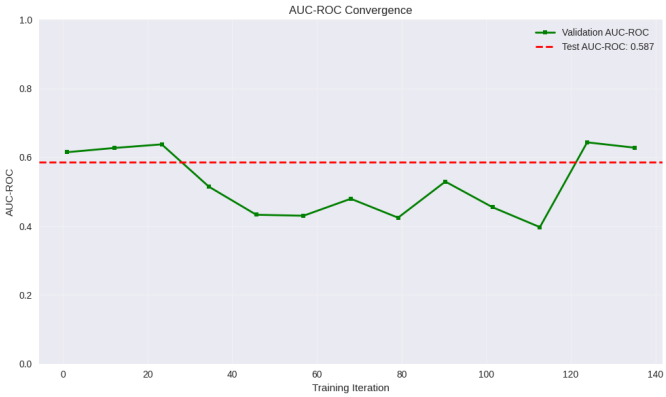

Fig. 9: GraphEPN: AUROC Convergence

These evaluation graphs were produced by GraphEPN using our optimal hyperparameters. The loss convergence graph provides training, testing and validation curves. It is showing similar trend with graphSAGE, where the model periodically overfits, then recovers. Furthermore, the AUROC convergence graph track the validation set AUC after every 10 intervals of training and is compared to test set AUROC curve. The validation AUC varies between 0.4 to 0.65 through training.

This suggests that the model's variability in performance on unseen data. Lastly the ROC curves comparison graph with test AUC 0.587 and validation AUC 0.628, lie above the random baseline, which suggests that the model was able to generalize on training data.

Hyperparameters	Best Run	Worst Run
Accuracy	0.6685	0.6050
AUROC	0.6025	0.4108
F1-Score (Epitope)	0.2128	0.1106
Learning Rate	3e-4	3e-4
Focal Alpha	0.6	0.25
Focal Gamma	1.0	1.0
Epitope Boost	3.0	5.0
Epochs per Graph	40	45
Patience	20	10

TABLE III: GraphSAGE Hyperparameter Comparison

This table provides the results of our ablation study on graphSAGE with two sets of hyperparameters. The data presented in the table suggests that impact of changing focal-alpha, epitope boost value, epochs per graph and patience. Focal alpha control the weight of importance given to epitopes and non-epitope. This is crucial as our dataset has class imbalance, hence this ensures that the model is assigning higher weight on correctly identifying epitopes which are important for antibody binding sites rather than being skewed towards to majority non-epitope class. Furthermore, focal gamma ensures that the model can capture hard to find epitopes, which are residues in complex structures or with ambiguous binding properties. Moreover, epitope boost provides additional class weighting, which should ensure false negatives are incur more loss, as false positives are more dangerous for biological applications. Further increasing epitope boost, degrades model performance which might suggest that over-amplification for the minority class which led to the model creating more false positives and lower F1-score. Decreasing the patience parameters, which controls early stopping also contributes to model degradation as it likely caused premature stopping during training, preventing the model from fully learning the complex spatial relationships in protein graphs. Additionally, lowering epoch per graph results in less overfitting for model performance.

Hyperparameters	Best Run	Second Run	Worst Run
Accuracy	0.6590	0.6677	0.5435
AUROC	0.5869	0.5430	0.5560
F1-Score (Epitope)	0.2209	0.2084	0.1843
Learning Rate	3e-5	3e-4	3e-4
Focal Alpha	0.6	0.6	0.4
Epitope Boost	3.0	5.0	3.0
Epochs per Graph	40	40	50

TABLE IV: GraphEPN Hyperparameter Comparison

This table provides the results of our ablation study on GraphEPN with varying hyperparameters, demonstrating the impact of architectural and training modifications. The focal alpha parameter remains crucial for addressing class imbalance, with the 0.6 value ensuring appropriate weighting between epitope and non-epitope residues. The significantly lower learning rate (3e-5 vs 3e-4 in GraphSAGE) enables more stable convergence of the attention weights and global context aggregation, which is critical given the model's increased complexity from concatenating local node features with global context features. The epitope boost parameter works synergistically with the attention mechanism, as over-amplification can cause the attention weights to become overly concentrated on minority class examples, leading to reduced generalization and higher false positive rates. The patience parameter is particularly important in GraphEPN due to the additional computational overhead from global context aggregation requiring more epochs for the attention mechanism to properly learn meaningful global representations.

VI. Conclusion

Our experimental results show that both models performed similarly with unbalanced data, with graphEPN slightly outperforming graphSAGE on the second run. We combined ESM-2 embeddings to capture evolutionary and sequence information; physicochemical properties(hydrophobicity, charge, volume); structural features along with secondary structure information(DSSP). Furthermore, we created comprehensive node representations that capture both local and global protein characteristics. We encountered difficulties with handling the severe class imbalance in our dataset. However, we implemented FocalLoss combined with epitope boost to counteract the effects of imbalance. We implemented early stopping, gradient clipping and validation splits to prevent overfitting while ensuring convergence. Moreover, we implemented thresholds to balance precision and recall for the minority epitope class. Our final model evaluation metrics suggest room for future improvement, reflecting the difficulty of epitope prediction. Furthermore, our models suffered from class imbalance which lowered evaluation scores. Hence, more advanced dataset balancing techniques might lead to improved scores and greater generalization on training data. Finally, ensemble methods might benefit our model's ability to learn the complex spatial relationships in protein structures.

References

[1] S. Choi and D. Kim, "B cell epitope prediction by capturing spatial clustering property of the epitopes using graph attention network," *Sci. Rep.*, vol. 14, no. 1, p. 27496, 2024.

[2] F. Jiang, Y. Guo, H. Ma, S. Na, W. An, B. Song, Y. Han, J. Gao, T. Wang, and J. Huang, "AlphaEpi: Enhancing B Cell Epitope Prediction with AlphaFold 3," in *Proc. 15th ACM Int. Conf. Bioinform., Comput. Biol. Health Inform.*, 2024, pp. 1–8.

[3] C. Liu, L. Denzler, Y. Chen, A. Martin, and B. Paige, "AsEP: Benchmarking deep learning methods for antibody-specific epitope prediction," *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 11700–11734, 2024.

[4] M. Nielsen, A. Eugster, M. F. Jensen, M. Goel, A. Tiffeau-Mayer, A. Pelissier, S. Valkiers, M. R. Martínez, B. Meynard-Piganeeau, V. Greiff, et al., "Lessons learned from the IMMREP23 TCR-epitope prediction challenge," *ImmunoInformatics*, vol. 16, p. 100045, 2024.

[5] M. Shahab, M. W. Iqbal, A. Ahmad, F. M. Alshabrmi, D.-Q. Wei, A. Khan, and G. Zheng, "Immunoinformatics-driven in silico vaccine design for Nipah virus (NPV): Integrating machine learning and computational epitope prediction," *Comput. Biol. Med.*, vol. 170, p. 108056, 2024.

[6] H. Xu, R. Hu, X. Dong, L. Kuang, W. Zhang, C. Tu, Z. Li, and Z. Zhao, "ImmuneApp for HLA-I epitope prediction and immunopeptidome analysis," *Nat. Commun.*, vol. 15, no. 1, p. 8926, 2024.

[7] Z. Yan, K. Kim, H. Kim, B. Ha, A. Gambiez, J. Bennett, M. F. de Almeida Mendes, R. Trevizani, J. Mahita, E. Richardson, et al., "Next-generation IEDB tools: a platform for epitope prediction and analysis," *Nucleic Acids Res.*, vol. 52, no. W1, pp. W526–W532, 2024.

[8] S. Lu, Y. Li, Q. Ma, X. Nan, and S. Zhang, "A structure-based B-cell epitope prediction model through combining local and global features," *Front. Immunol.*, vol. 13, p. 890943, 2022.

[9] M. Collatz, F. Mock, E. Barth, M. Hölzer, K. Sachse, and M. Marz, "EpiDope: a deep neural network for linear B-cell epitope prediction," *Bioinformatics*, vol. 37, no. 4, pp. 448–455, 2020.

[10] W. Alghamdi, M. Attique, E. Alzahrani, M. Z. Ullah, and Y. D. Khan, "LBCEPred: a machine learning model to predict linear B-cell epitopes," *Brief. Bioinform.*, vol. 23, no. 3, p. bbac035, 2022.

[11] F. Wang, X. Dai, L. Shen, and S. Chang, "GraphEPN: A deep learning framework for B-cell epitope prediction leveraging graph neural networks," *Appl. Sci.*, vol. 15, no. 4, p. 2159, 2025.

[12] T. Qiu, L. Zhang, Z. Chen, Y. Wang, T. Mao, C. Wang, Y. Cun, G. Zheng, D. Yan, M. Zhou, K. Tang, and Z. Cao, "SEPPA-mAb: spatial epitope prediction of protein antigens for mAbs," *Nucleic Acids Res.*, vol. 51, no. W1, pp. W528–W534, 2023.

[13] M. H. Høie, F. S. Gade, J. M. Johansen, C. Würtzen, O. Winther, M. Nielsen, and P. Marcatili, "DiscoTope-3.0: improved B-cell epitope prediction using inverse folding latent representations," *Front. Immunol.*, vol. 15, p. 1322712, 2024.

[14] B. M. da Silva, Y. Myung, D. B. Ascher, and D. E. V. Pires, "epitope3D: a machine learning method for conformational B-cell epitope prediction," *Brief. Bioinform.*, vol. 23, no. 1, p. bbab423, 2022.

[15] B. M. da Silva, D. B. Ascher, and D. E. V. Pires, "epitope1D: accurate taxonomy-aware B-cell linear epitope prediction," *Brief. Bioinform.*, vol. 24, no. 3, p. bbad114, 2023.

[16] J. Liu, G. P. Ong, and X. Chen, "GraphSAGE-based traffic speed forecasting for segment network with sparse data," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 3, pp. 1755–1766, 2022.